Nhill — API v1 reference

Français : ../fr/reference-api.md

Generated from the server's code by `scripts/docs/gen_api_reference.py` (do not edit by hand: change the code or `scripts/docs/api_descriptions.py`, then run the script again; continuous integration checks that this file is up to date). Version 3.0.0: 198 endpoints in normal operation and 5 in setup mode.

Conventions

- Base: `https://<server address>/api/v1`; the address is entered in the applications, none is compiled into them. UTF-8 JSON. Language of messages and PDFs: `Accept-Language` header (`en…` → English, otherwise French).
- Authentication: `Authorization: Bearer <access token>` (15 minutes by default, `NH_ACCESS_TOKEN_MINUTES`), renewed by `POST /api/v1/auth/refresh`. The console uses a separate administration token that cannot be swapped.
- Amounts: decimal strings (`"1234.50"`); quantities: strings (`"2.5"`); rates: integer basis points (`1600` = 16 %). Dates `YYYY-MM-DD`, timestamps ISO 8601 UTC.
- "Idem." column: the `Idempotency-Key` header (UUID) is required (`400 idempotency_key_required`); a replay returns the same answer with `Idempotent-Replayed: true`.
- Pagination: `limit` (1 to 200), `offset`. Reports: `from`, `to`, `format=json|csv|pdf`.
- Errors: `{"error": {"code", "message", "details", "request_id"}}`.

Access checks

- **Public**: no token. **User**: member of a business. **Console**: platform administrator ("Console roles" column; `superadmin` has every right). **Webhook**: provider signature. **Setup**: server without a database, setup token printed in its log.
- "Permission": right the role must have (table below); otherwise `403 permission_denied`.
- "Plan": feature the business's plan must include (`a|b`: either); otherwise `403 feature_not_included`. A `pos.*`, `purchases.*` or `expenses.*` permission also requires the matching feature. Some routes also check a feature from the content: document type (quotes and orders → `sales.quotes`, invoices and credit notes → `sales.invoices`), a currency other than the main currency (`multi_currency`), a warehouse other than the main one (`stock.multi_warehouse`). Lists only return data of the included features.
- "Write": writing allowed only with full access; read-only subscription → `402 subscription_suspended`; business suspended by the platform → `403 establishment_suspended`. Closed business or no access → `403 establishment_closed` on every user route.
- Plan limits (users, tills, warehouses, products, documents per month, storage): exceeded → `409 plan_limit_reached` (`user_limit_reached` for users, `plan_register_limit` for tills), with `limit_key`, `limit` and `used` in `details`.

Permissions per role

Role	Permissions
Owner (`owner`)	`settings.manage`, `users.manage`, `code.view`, `code.rotate`, `subscription.manage`, `catalog.write`, `parties.write`, `sales.write`, `sales.issue`, `purchases.write`, `stock.adjust`, `expenses.write`, `payments.write`, `reports.read`, `audit.read`, `pos.sell`, `pos.refund`, `pos.void`, `pos.cash`, `pos.supervise`, `pos.manage`, `pos.report`
Administrator (`admin`)	`settings.manage`, `users.manage`, `code.view`, `code.rotate`, `subscription.manage`, `catalog.write`, `parties.write`, `sales.write`, `sales.issue`, `purchases.write`, `stock.adjust`, `expenses.write`, `payments.write`, `reports.read`, `audit.read`, `pos.sell`, `pos.refund`, `pos.void`, `pos.cash`, `pos.supervise`, `pos.manage`, `pos.report`
Manager (`manager`)	`code.view`, `catalog.write`, `parties.write`, `sales.write`, `sales.issue`, `purchases.write`, `stock.adjust`, `expenses.write`, `payments.write`, `reports.read`, `pos.sell`, `pos.refund`, `pos.supervise`, `pos.report`
Salesperson (`sales`)	`parties.write`, `sales.write`, `sales.issue`, `payments.write`, `pos.sell`
Accountant (`accountant`)	`expenses.write`, `payments.write`, `reports.read`, `audit.read`, `pos.report`
Read-only (`viewer`)	no write access
Till supervisor (`cash_manager`)	`pos.sell`, `pos.refund`, `pos.void`, `pos.cash`, `pos.supervise`, `pos.report`
Cashier (`cashier`)	`pos.sell`

Setup mode (server without a database)

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/health`	Setup	—	—	—	—	—	Server waiting for installation: `{"status":"setup","database":false}`
GET	`/api/v1/meta`	Setup	—	—	—	—	—	Server identity (product, version, API, minimum client version) with `mode: setup`; the applications use it to check an address before saving it
GET	`/api/v1/setup/status`	Setup	—	—	—	—	—	`setup` or `restarting` (installation done, restart in progress) and version
POST	`/api/v1/setup/check`	Setup	—	—	—	—	—	Tests the database connection `{setup_token, database:{…}}` without writing anything; token printed in the server log, valid 60 min, 20 attempts per 15 min per IP address
POST	`/api/v1/setup/apply`	Setup	—	—	—	—	—	Installs: saves the configuration, applies the migrations, optionally creates the first administrator `{first_admin:{email, full_name}}` (first-access token returned once), then the server restarts in normal mode; `409 already_installed` afterwards

Service and reference data

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/health`	Public	—	—	—	—	—	Service and database status (200 `ok`, 503 degraded)
GET	`/api/v1/meta`	Public	—	—	—	—	—	Version, minimum client version, countries, currencies (36), languages
GET	`/api/v1/countries/:cc/profile`	Public	—	—	—	—	—	Country profile: legal identifiers, taxes, notices, numbering

Sign-up and joining

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
POST	`/api/v1/establishments`	Public	—	—	—	—	—	Creates a business: business, owner, assigned plan (trial or assignment rule) and code; refuses a duplicate legal identifier (`establishment_exists`)
POST	`/api/v1/establishments/attachment-requests`	Public	—	—	—	—	—	Request to join an existing company (same answer whether the company exists or not)
POST	`/api/v1/join/check-code`	Public	—	—	—	—	—	Join, step 1: business code → 10-minute ticket (generic error, rate-limited per IP address)
POST	`/api/v1/join/login`	Public	—	—	—	—	—	Join: ticket + e-mail + password of a member
POST	`/api/v1/join/activate`	Public	—	—	—	—	—	Join: ticket + e-mail + invitation code → account and session created

Authentication and account

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
POST	`/api/v1/auth/login`	Public	—	—	—	—	—	Login (e-mail, password, business when several). With a temporary pass `{"password_change_required":true,"change_token","expire` instead of a session; expired temporary password → `401 temporary_password_expired`
POST	`/api/v1/auth/refresh`	Public	—	—	—	—	—	Refresh-token rotation (reuse detected → session revoked)
POST	`/api/v1/auth/logout`	User	—	—	—	—	—	Closes the current session
GET	`/api/v1/me`	User	—	—	—	—	—	User, role, permissions, rights version, business, subscription (details for owner and administrators, access state for the others), plan rights and us other businesses
PATCH	`/api/v1/me`	User	—	—	—	—	—	Name, phone, language
POST	`/api/v1/me/password`	User	—	—	—	—	—	Password change (closes the other sessions)
POST	`/api/v1/auth/password/forgot`	Public	—	—	—	—	—	Forgotten password `{email}` → **always** `202`. 8-digit code by e-mail fo owner, an administrator or an account of several businesses; “contact yo administrator” message for an employee of a single business; nothing fo unknown address. 3 requests per hour per address, 10 per hour per IP
POST	`/api/v1/auth/password/reset`	Public	—	—	—	—	—	`{email, code, new_password}` → 204; wrong, expired or used-up c `400 invalid_reset_code` (same error in every case); every session i
POST	`/api/v1/auth/password/change-required`	Public	—	—	—	—	—	`{change_token, new_password}` after a login with a temporary pass single-use 15-minute ticket (`400 invalid_change_token`); `422 password_reused` if identical to the temporary one; same answer as a
POST	`/api/v1/me/switch`	User	—	—	—	—	—	Switch business without typing the password again
GET	`/api/v1/notifications`	User	—	—	—	—	—	The user’s notifications
POST	`/api/v1/notifications/:id/read`	User	—	—	—	—	—	Mark a notification as read
POST	`/api/v1/members/:id/password-reset`	User	`users.manage`	—	—	—	—	Temporary password for a member of **this** business, shown only once; re for oneself, the owner, an administrator if the author is not the owner, a deactivated member (`409 member_disabled`), an account active in an business (`409 account_shared`); 10 per 15 min per author
GET	`/api/v1/members`	User	`users.manage`	—	—	—	—	Members, pending invitations, user limit of the plan
PATCH	`/api/v1/members/:id`	User	`users.manage`	—	—	—	—	Change a member’s role or deactivate them (sessions revoked)

Business, team and rights

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/entitlements`	User	—	—	—	—	—	The business’s rights: features included in the plan (never the others), limits and usage, FR/EN labels, rights version
GET	`/api/v1/establishment`	User	—	—	—	—	—	Business record, settings, compliance notice
PATCH	`/api/v1/establishment`	User	`settings.manage`	—	yes	—	—	Edit the record

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
PUT	`/api/v1/establishment/legal-ids`	User	`settings.manage`	—	yes	—	—	Replace the legal identifiers (validated against the country profile)
PATCH	`/api/v1/establishment/settings`	User	`settings.manage`	—	yes	—	—	Document settings: language, payment terms, tax rounding, footer, payment instructions, notices (FR/EN), negative stock
PUT	`/api/v1/establishment/logo`	User	`settings.manage`	—	yes	—	—	Upload the logo (PNG or JPEG, raw body, actual content checked)
GET	`/api/v1/establishment/logo`	User	—	—	—	—	—	Read the logo
DELETE	`/api/v1/establishment/logo`	User	`settings.manage`	—	yes	—	—	Delete the logo
GET	`/api/v1/establishment/code`	User	`code.view`	—	—	—	—	Current business code and renewal date
POST	`/api/v1/establishment/code/rotate`	User	`code.rotate`	—	—	—	—	Renew the code now
GET	`/api/v1/establishment/audit`	User	`audit.read`	—	—	—	—	Raw log (endpoint kept; prefer `/api/v1/activity`)
GET	`/api/v1/establishment/export`	User	—	—	—	—	—	Full ZIP archive of the business's data (JSON, CSV, attachments, manifest with SHA-256 hashes); owner only, even with a suspended or expired subscription; 3 per 15 min
POST	`/api/v1/invitations`	User	`users.manage`	—	yes	—	—	Invite (e-mail, role) within the user limit; activation code returned only once
DELETE	`/api/v1/invitations/:id`	User	`users.manage`	—	—	—	—	Revoke an invitation

Subscription and payment

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/plans`	Public	—	—	—	—	—	Published plans with their prices per duration and currency, limits and features
GET	`/api/v1/subscription`	User	`subscription.manage`	—	—	—	—	Subscription (plan, price, due date, scheduled change, failures), orders, history
POST	`/api/v1/subscription/quote`	User	`subscription.manage`	—	—	—	—	Server-computed quote `{price_id, promo_code}` → kind (new, renewal, upgrade, downgrade), price, discount, prorated credit, total
POST	`/api/v1/subscription/orders`	User	`subscription.manage`	—	—	—	—	Create the order (`price_id`, `promo_code`, `provider`); zero total → applied at once
POST	`/api/v1/subscription/orders/:id/checkout`	User	`subscription.manage`	—	—	—	—	Create the payment session with the provider → `checkout_url`
POST	`/api/v1/subscription/orders/:id/cancel`	User	`subscription.manage`	—	—	—	—	Cancel an unpaid order (promo code released)
POST	`/api/v1/subscription/auto-renew`	User	`subscription.manage`	—	—	—	—	Give or withdraw express consent to automatic renewal
GET	`/payment/return`	Public	—	—	—	—	—	HTML return page after payment (activates nothing)
POST	`/api/v1/webhooks/payments/:provider`	Webhook	—	—	—	—	—	Payment provider notification: signature checked, deduplicated, re-read from the provider

Support

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/support/tickets`	User	—	—	—	—	—	Support requests: one's own; all of the business's with `users.manage`
POST	`/api/v1/support/tickets`	User	—	—	—	—	—	New request `{category: bug|question|billing|feature, subject, body}` (20 per day)
GET	`/api/v1/support/tickets/:id`	User	—	—	—	—	—	Request and messages (the team's internal notes excluded)
POST	`/api/v1/support/tickets/:id/messages`	User	—	—	—	—	—	Reply
POST	`/api/v1/support/tickets/:id/close`	User	—	—	—	—	—	Close the request
POST	`/api/v1/client-errors`	User	—	—	—	—	—	Technical error report from an application (personal data masked)

Catalogue

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/tax-rates`	User	—	—	—	—	—	Tax rates
POST	`/api/v1/tax-rates`	User	`settings.manage`	—	yes	—	—	Create a rate
PATCH	`/api/v1/tax-rates/:id`	User	`settings.manage`	—	yes	—	—	Label, active, default (the rate itself never changes)
GET	`/api/v1/product-categories`	User	—	—	—	—	—	Product categories
POST	`/api/v1/product-categories`	User	`catalog.write`	—	yes	—	—	Create a category
DELETE	`/api/v1/product-categories/:id`	User	`catalog.write`	—	yes	—	—	Delete an empty category
GET	`/api/v1/products`	User	—	—	—	—	—	Products and services (search, category, low stock, pagination)
GET	`/api/v1/products/export.csv`	User	—	—	—	—	—	Catalogue CSV export
GET	`/api/v1/products/:id`	User	—	—	—	—	—	Product
POST	`/api/v1/products`	User	`catalog.write`	—	yes	—	—	Create a product or service (within the plan's product limit)
PATCH	`/api/v1/products/:id`	User	`catalog.write`	—	yes	—	—	Edit a product
POST	`/api/v1/products/import`	User	`catalog.write`	`catalog.import`	yes	—	—	All-or-nothing CSV import (errors per line)

Customers and suppliers

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/customers`	User	—	—	—	—	—	Customers (search, balances)
GET	`/api/v1/suppliers`	User	—	`purchases|expenses`	—	—	—	Suppliers
GET	`/api/v1/parties/:id`	User	—	—	—	—	—	Party, balances, documents
POST	`/api/v1/parties`	User	`parties.write`	—	yes	—	—	Create a customer and/or supplier
PATCH	`/api/v1/parties/:id`	User	`parties.write`	—	yes	—	—	Edit a party

Sales documents

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/sales-documents`	User	—	—	—	—	—	Documents (type, status, party, period, search, overdue); only the types included in the plan
GET	`/api/v1/sales-documents/:id`	User	—	—	—	—	—	Full document, payments, history
GET	`/api/v1/sales-documents/:id/pdf`	User	—	—	—	—	—	Document PDF: `format=a4|ticket`, `width=58|80` (the same document in both layouts; `inline` for display)
POST	`/api/v1/sales-documents`	User	`sales.write`	—	yes	yes	—	Create a draft (idempotent)
PUT	`/api/v1/sales-documents/:id`	User	`sales.write`	—	yes	—	—	Edit a draft
DELETE	`/api/v1/sales-documents/:id`	User	`sales.write`	—	yes	—	—	Delete a draft (never an issued document)
POST	`/api/v1/sales-documents/:id/issue`	User	`sales.issue`	—	yes	yes	—	Issue: final number, stock, frozen exchange rate, PDF hash (idempotent; monthly document limit, never for a credit note)
POST	`/api/v1/sales-documents/:id/status`	User	`sales.write`	—	yes	—	—	Controlled transition (quote accepted or declined, order cancelled)
POST	`/api/v1/sales-documents/:id/convert`	User	`sales.write`	—	yes	yes	—	Quote or order → order or invoice (draft)
POST	`/api/v1/sales-documents/:id/credit-note`	User	`sales.issue`	—	yes	yes	—	Credit-note draft for an issued invoice (restock option)
PUT	`/api/v1/sales-documents/:id/fiscal-reference`	User	`sales.issue`	—	yes	—	—	Record the reference of the standardised invoice issued by the approved device (reconciliation, no certification)

Purchases

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/purchases`	User	—	`purchases`	—	—	—	Purchases
GET	`/api/v1/purchases/:id`	User	—	`purchases`	—	—	—	Purchase
POST	`/api/v1/purchases`	User	`purchases.write`	`purchases`	yes	yes	—	Create a draft purchase (idempotent)
PUT	`/api/v1/purchases/:id`	User	`purchases.write`	`purchases`	yes	—	—	Edit a draft

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
POST	`/api/v1/purchases/:id/receive`	User	`purchases.write`	`purchases`	yes	yes	—	Receive: stock entries, purchase prices updated (idempotent)
POST	`/api/v1/purchases/:id/cancel`	User	`purchases.write`	`purchases`	yes	—	—	Cancel a draft

Stock

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/warehouses`	User	—	—	—	—	—	Warehouses (and whether several warehouses are allowed)
POST	`/api/v1/warehouses`	User	`stock.adjust`	`stock.multi_warehouse`	yes	—	—	Create a warehouse (within the plan's warehouse limit)
GET	`/api/v1/stock`	User	—	—	—	—	—	Stock levels and estimated value
GET	`/api/v1/stock/movements`	User	—	—	—	—	—	Stock movements
POST	`/api/v1/stock/adjustments`	User	`stock.adjust`	—	yes	yes	—	Adjustment or opening stock, reason required (idempotent)
POST	`/api/v1/stock/transfers`	User	`stock.adjust`	`stock.multi_warehouse`	yes	yes	—	Transfer between warehouses (idempotent)

Expenses

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/expense-categories`	User	—	`expenses`	—	—	—	Expense categories
POST	`/api/v1/expense-categories`	User	`expenses.write`	`expenses`	yes	—	—	Create a category
GET	`/api/v1/expenses`	User	—	`expenses`	—	—	—	Expenses (period, category, unpaid)
POST	`/api/v1/expenses`	User	`expenses.write`	`expenses`	yes	yes	—	Create an expense, possibly paid (idempotent)
PUT	`/api/v1/expenses/:id/receipt`	User	`expenses.write`	`expenses`	yes	—	—	Attach a receipt (PDF, PNG or JPEG, 5 MB; storage limit of the plan)
GET	`/api/v1/expenses/:id/receipt`	User	—	`expenses`	—	—	—	Read the receipt

Payments, cash accounts and exchange rates

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/payments`	User	—	—	—	—	—	Payments (direction, period)
POST	`/api/v1/payments`	User	`payments.write`	—	yes	yes	—	Payment of an invoice, a credit note (refund), a purchase or an expense — never more than the amount due (idempotent)
GET	`/api/v1/cash-accounts`	User	—	—	—	—	—	Cash and bank accounts, recorded balance
POST	`/api/v1/cash-accounts`	User	`settings.manage`	—	yes	—	—	Create a cash or bank account
GET	`/api/v1/exchange-rates`	User	—	`multi_currency`	—	—	—	Recorded exchange rates
POST	`/api/v1/exchange-rates`	User	`settings.manage`	`multi_currency`	yes	—	—	Record a rate (date and source)

Till (Nhill POS)

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/pos/config`	User	`pos.sell`	`pos`	—	—	—	Accessible tills, payment methods, rules, permissions, the user's open session
GET	`/api/v1/pos/catalog`	User	`pos.sell`	`pos`	—	—	—	`?register_id=`: active products with price, tax and stock of the till's warehouse
GET	`/api/v1/pos/registers`	User	`pos.report`	`pos`	—	—	—	Tills, payment methods, rules; till limit of the plan
POST	`/api/v1/pos/registers`	User	`pos.manage`	`pos`	yes	—	—	Create a till (code, name, location, warehouse) within the plan's till limit
PATCH	`/api/v1/pos/registers/:id`	User	`pos.manage`	`pos`	yes	—	—	Edit, activate or deactivate a till
PUT	`/api/v1/pos/registers/:id/users`	User	`pos.manage`	`pos`	yes	—	—	Authorised cashiers `{user_ids}` (active members only)
POST	`/api/v1/pos/payment-methods`	User	`pos.manage`	`pos`	yes	—	—	Add a payment method
PATCH	`/api/v1/pos/payment-methods/:id`	User	`pos.manage`	`pos`	yes	—	—	Edit, activate or deactivate a payment method

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
PATCH	`/api/v1/pos/settings`	User	`pos.manage`	`pos`	yes	—	—	`max_discount_bp`, `allow_price_override`, `receipt_footer`, `receipt_width_mm` (58 or 80)
GET	`/api/v1/pos/promo-codes`	User	`pos.manage`	`pos`	—	—	—	The business's sales promo codes and number of uses
POST	`/api/v1/pos/promo-codes`	User	`pos.manage`	`pos`	yes	—	—	Create a sales promo code (percentage, minimum amount, validity, maximum uses)
PATCH	`/api/v1/pos/promo-codes/:id`	User	`pos.manage`	`pos`	yes	—	—	Change the validity or activate / deactivate a code
POST	`/api/v1/pos/approvals`	User	`pos.sell`	`pos`	—	—	—	Supervisor approval `{email, password, action, session_id}` → single-use token (3 min)
POST	`/api/v1/pos/sessions`	User	`pos.sell`	`pos`	yes	—	—	Open a session `{register_id, opening_float}`
GET	`/api/v1/pos/sessions`	User	—	`pos`	—	—	—	Till sessions: one's own, all with `pos.report` (`register_id` filter, pagination)
GET	`/api/v1/pos/sessions/:id`	User	—	`pos`	—	—	—	Session detail: sales and returns, takings per method, taxes, cash movements, expected cash and difference, documents issued
POST	`/api/v1/pos/sessions/:id/cash-movements`	User	`pos.sell`	`pos`	yes	yes	—	Cash in or out (out: `pos.cash` or approval)
POST	`/api/v1/pos/sessions/:id/close`	User	`pos.sell`	`pos`	yes	yes	—	Closing `{counted: {method: amount}, note}` (cash required)
GET	`/api/v1/pos/sessions/:id/report.pdf`	User	—	`pos`	—	—	—	X reading (open session) or Z report (closed session)
POST	`/api/v1/pos/sales`	User	`pos.sell`	`pos`	yes	yes	—	Sale `{session_id, lines, document: receipt|invoice, customer_id?, promo_code?, payments, approval_token?}` → `{receipt, change}`; an invoice requires an identified customer
GET	`/api/v1/pos/sales/:id/receipt.pdf`	User	`pos.sell`	`pos`	—	yes	—	Sale PDF: 58/80 mm receipt or A4 invoice, same template as Nhill Business
POST	`/api/v1/pos/sales/:id/return`	User	`pos.sell`	`pos`	yes	—	—	Return `{session_id, reason, lines[{line_no, quantity}], refunds?, restock}` (`pos.refund` or approval)
POST	`/api/v1/pos/sales/:id/void`	User	`pos.sell`	`pos`	yes	—	—	Void the receipt within its session (`pos.void` or approval)
GET	`/api/v1/pos/receipts`	User	`pos.sell`	`pos`	—	—	—	`?number=`: find a till receipt or invoice by its number

Reports and activity log

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/activity`	User	—	—	—	—	—	Activity log: `user_id`, `action` (prefix), `from`, `to` filters, pagination, `format=csv|pdf`; without `audit.read`, one's own actions only
GET	`/api/v1/reports/staff`	User	`audit.read`	`reports.advanced`	—	—	—	Activity per employee and per currency (`from`, `to`, `format`)
GET	`/api/v1/reports/dashboard`	User	—	—	—	—	—	Dashboard (`from`, `to`); personal scope without `reports.read`; indicators limited to the plan's features
GET	`/api/v1/reports/sales`	User	`reports.read`	`sales.invoices|pos`	—	—	—	Sales journal (with "Issued by")
GET	`/api/v1/reports/expenses`	User	`reports.read`	`expenses`	—	—	—	Expenses
GET	`/api/v1/reports/stock-movements`	User	`reports.read`	—	—	—	—	Stock movements
GET	`/api/v1/reports/receivables`	User	`reports.read`	`sales.invoices`	—	—	—	Customer receivables to date
GET	`/api/v1/reports/payables`	User	`reports.read`	`purchases|expenses`	—	—	—	Payables to date
GET	`/api/v1/reports/payments`	User	`reports.read`	—	—	—	—	Payments (with "Recorded by")
GET	`/api/v1/reports/products`	User	`reports.read`	`reports.advanced`	—	—	—	Sales per product, estimated margin

Console — administrator access

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
POST	`/api/v1/admin/auth/setup`	Public	—	—	—	—	—	First access: setup token + password (at least 8 characters, not a too-common password) → TOTP secret
POST	`/api/v1/admin/auth/setup/confirm`	Public	—	—	—	—	—	First access: TOTP code → account active
POST	`/api/v1/admin/auth/login`	Public	—	—	—	—	—	Login: e-mail + password + TOTP code (code cannot be replayed)
POST	`/api/v1/admin/auth/logout`	Console	—	—	—	—	—	Close the administration session
GET	`/api/v1/admin/me`	Console	—	—	—	—	—	Current administrator

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
POST	`/api/v1/admin/admins`	Console	—	—	—	—	`superadmin`	Invite an administrator; setup token shown only once

Console — businesses

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/admin/establishments`	Console	—	—	—	—	—	List of businesses (search, subscription state, `erasable=true` : businesses to erase)
GET	`/api/v1/admin/establishments/:id`	Console	—	—	—	—	—	Platform record: subscription, codes (without thei value), 30-day usage, erasure state
POST	`/api/v1/admin/establishments/:id/suspend`	Console	—	—	—	—	`support`	Suspend with a reason; sessions revoked
POST	`/api/v1/admin/establishments/:id/reactivate`	Console	—	—	—	—	`support`	Reactivate with a reason
POST	`/api/v1/admin/establishments/:id/erase`	Console	—	—	—	—	`superadmin`	Permanent erasure `{reason, confirm_trade_name, customer_request}` : possible 90 days after access ends, earlier only at customer's request and never during full access (`not_erasable`); paid orders archived 10 years; recorded in the platform log
POST	`/api/v1/admin/establishments/:id/codes/reissue`	Console	—	—	—	—	`support`	Reissue the code with a reason; the code is not sh to the administrator
GET	`/api/v1/admin/attachment-requests`	Console	—	—	—	—	`support`	Join requests
POST	`/api/v1/admin/attachment-requests/:id/resolve`	Console	—	—	—	—	`support`	Accept or refuse with a note
GET	`/api/v1/admin/establishments/:id/stats`	Console	—	—	—	—	`billing` , `auditor`	**Aggregated** statistics over a period; consultation logged and visible to the business
POST	`/api/v1/admin/establishments/:id/subscription`	Console	—	—	—	—	`billing`	`{action: extend|suspend|reactivate|cancel|set_p reason}` — reason required, logged, business not
POST	`/api/v1/admin/establishments/:id/orders`	Console	—	—	—	—	`billing`	Create a payment request for a business
GET	`/api/v1/admin/establishments/:id/entitlements`	Console	—	—	—	—	—	The business's effective rights (every feature, on c and usage counters, never content

Console — plans and rights

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/admin/plans`	Console	—	—	—	—	—	Plans
POST	`/api/v1/admin/plans`	Console	—	—	—	—	`billing`	Create a plan (code, FR/EN names and descriptions, audience, tier, limits, features)
PATCH	`/api/v1/admin/plans/:id`	Console	—	—	—	—	`billing`	Edit a plan and its terms (trial, grace period, reminders, self-service subscription, automatic renewal, fallback plan at expiry, archiving); applies to future requests
POST	`/api/v1/admin/plans/:id/prices`	Console	—	—	—	—	`billing`	Add or replace a price (1, 3, 6, 12, 24 or 36 months, currency)
PATCH	`/api/v1/admin/plan-prices/:id`	Console	—	—	—	—	`billing`	Edit or withdraw a price
GET	`/api/v1/admin/capabilities`	Console	—	—	—	—	—	Catalogue of features and limits (the list follows the server's code)
PATCH	`/api/v1/admin/capabilities/:key`	Console	—	—	—	—	`superadmin`	Edit the FR/EN labels and descriptions of a feature or limit
GET	`/api/v1/admin/plans/:id/entitlements`	Console	—	—	—	—	—	The plan's value for each feature and limit, with the catalogue
PUT	`/api/v1/admin/plans/:id/entitlements`	Console	—	—	—	—	`billing`	Replace every value of the plan (a missing entry takes the catalogue default); subscribed businesses get their new rights at their next request; before/after logged
GET	`/api/v1/admin/plan-rules`	Console	—	—	—	—	—	Plan assignment rules
POST	`/api/v1/admin/plan-rules`	Console	—	—	—	—	`billing`	Create a rule `{plan_id, mode: trial|grant, audience, country_code, duration_days, priority}` ; an open-ended grant is reserved to free or custom plans
PATCH	`/api/v1/admin/plan-rules/:id`	Console	—	—	—	—	`billing`	Edit a rule
DELETE	`/api/v1/admin/plan-rules/:id`	Console	—	—	—	—	`billing`	Delete a rule

Console — billing

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/admin/orders`	Console	—	—	—	—	—	Subscription payment requests
POST	`/api/v1/admin/orders/:id/confirm-manual`	Console	—	—	—	—	`billing`	Manual confirmation of a transfer: reference + reason, logged

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/admin/webhook-events`	Console	—	—	—	—	`billing`, `auditor`	Events received from providers
GET	`/api/v1/admin/transactions`	Console	—	—	—	—	`billing`, `auditor`	Subscription transactions (status, provider, business, period, pagination)
GET	`/api/v1/admin/transactions/:id`	Console	—	—	—	—	`billing`, `auditor`	Transaction detail and provider references
GET	`/api/v1/admin/payment-providers`	Console	—	—	—	—	`billing`	Configured payment providers (secrets never returned) and public return address
POST	`/api/v1/admin/payment-providers`	Console	—	—	—	—	`superadmin`	Add a provider (`stripe`, `paypal`, `payoneer` without adapter, `generic_hmac`); secrets encrypted
PATCH	`/api/v1/admin/payment-providers/:id`	Console	—	—	—	—	`superadmin`	Edit a provider, its secrets or its activation
GET	`/api/v1/admin/promo-codes`	Console	—	—	—	—	`billing`	Subscription promo codes
POST	`/api/v1/admin/promo-codes`	Console	—	—	—	—	`billing`	Create a code (percentage or fixed amount in a currency)
PATCH	`/api/v1/admin/promo-codes/:id`	Console	—	—	—	—	`billing`	Edit validity, limits or activation (code and discount cannot change)
GET	`/api/v1/admin/promo-codes/:id/redemptions`	Console	—	—	—	—	`billing`, `auditor`	Uses of a code

Console — support and operations

Method	Path	Access	Permission	Plan	Write	Idem.	Console roles	Description
GET	`/api/v1/admin/audit`	Console	—	—	—	—	`auditor`, `support`	Platform log; details of business actions masked
GET	`/api/v1/admin/stats`	Console	—	—	—	—	—	Platform-wide figures
GET	`/api/v1/admin/announcements`	Console	—	—	—	—	—	Announcements
POST	`/api/v1/admin/announcements`	Console	—	—	—	—	`support`	Publish an announcement
GET	`/api/v1/admin/settings`	Console	—	—	—	—	—	Platform settings
PUT	`/api/v1/admin/settings/:key`	Console	—	—	—	—	`superadmin`	Change a setting from the allowed list: `trial_days` 0–365, `grace_days` 0–60, `reminder_days_before` 1–60, `code_rotation_days` 1–365, `code_check_max_per_15min` and `login_max_per_15min` 3–100, `invitation_days` 1–60, `trial_max_users` 1–1000, `password_reset_minutes` 5–60, `password_reset_max_attempts` 1–10, `temporary_password_hours` 1–168, `payment_instructions` (FR/EN object)
GET	`/api/v1/admin/reports/establishments`	Console	—	—	—	—	`billing`, `auditor`	Platform report: one row per business and per currency (JSON, CSV, PDF)
GET	`/api/v1/admin/reports/establishments/:id`	Console	—	—	—	—	`billing`, `auditor`	Aggregated monthly report of a business (JSON, CSV, PDF)
GET	`/api/v1/admin/tickets`	Console	—	—	—	—	`support`	Support requests of every business (status, category, business, open)
GET	`/api/v1/admin/tickets/:id`	Console	—	—	—	—	`support`	Request and messages, internal notes included
POST	`/api/v1/admin/tickets/:id/messages`	Console	—	—	—	—	`support`	Reply, or add an internal note the business cannot see
PATCH	`/api/v1/admin/tickets/:id`	Console	—	—	—	—	`support`	Status, severity, assignee
GET	`/api/v1/admin/errors`	Console	—	—	—	—	`support`, `auditor`	Grouped technical errors
POST	`/api/v1/admin/errors/:id/resolve`	Console	—	—	—	—	`support`	Mark an error as resolved
GET	`/api/v1/admin/notifications`	Console	—	—	—	—	—	Console notifications
POST	`/api/v1/admin/notifications/read`	Console	—	—	—	—	—	Mark everything as read
GET	`/api/v1/admin/overview`	Console	—	—	—	—	—	Operating indicators (subscriptions, payments, requests, errors)

Summary

Section	Endpoints
Setup mode (server without a database)	5
Service and reference data	3
Sign-up and joining	5
Authentication and account	15

Section	Endpoints
Business, team and rights	14
Subscription and payment	9
Support	6
Catalogue	12
Customers and suppliers	5
Sales documents	11
Purchases	6
Stock	6
Expenses	6
Payments, cash accounts and exchange rates	6
Till (Nhill POS)	24
Reports and activity log	10
Console — administrator access	6
Console — businesses	12
Console — plans and rights	13
Console — billing	12
Console — support and operations	17
Total	**203**